



A Hands-on Introduction to Building Multimodal Agents

Antonio Rueda-Toicen

Senior Developer Relations Manager



Agentic AI Summit | Berlin, 2026

Agenda

- **Intro to LLM-based agents**
- **Intro to Multimodality**
- **Intuitions about embeddings and decoders**
- **Overview of Nemotron 3 Nano Omni**
- **Providing context with Nemotron Parse 2.0**
- **Experimenting with the Notebooks**



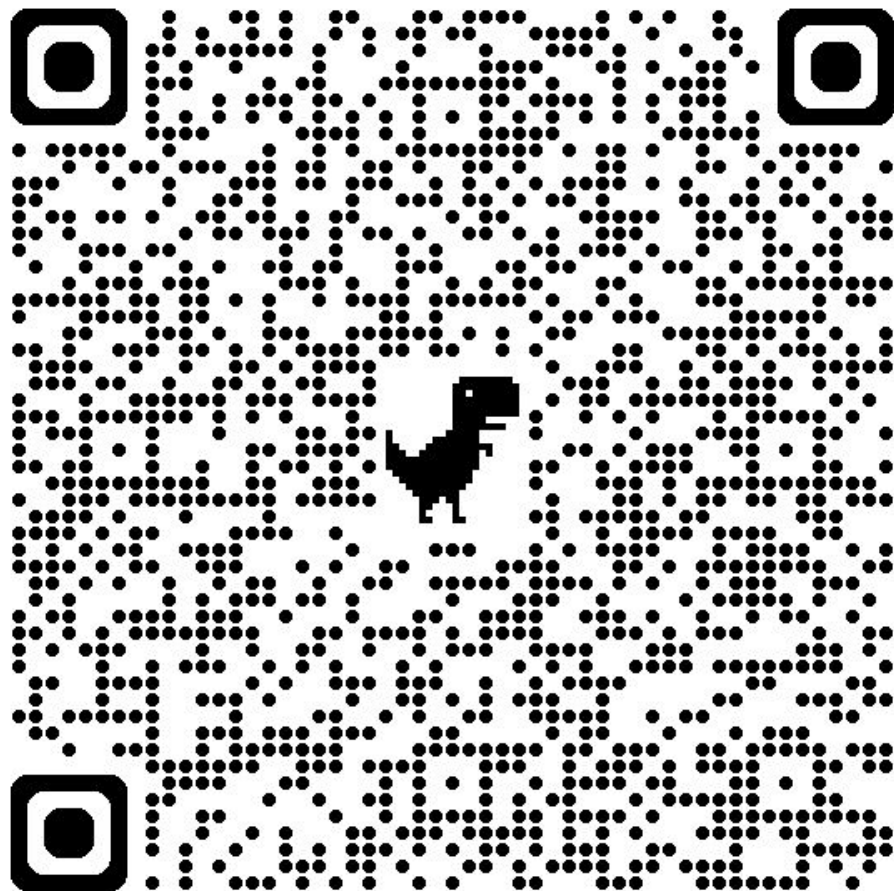
Notebooks

- [Using Nemotron models through Nvidia Inference Microservices](#)
- [Using Nemotron 3 Omni + Nemotron Parse 1.2 for document understanding](#)
- [A Nemotron + LangGraph agent for word counting](#)

Setup

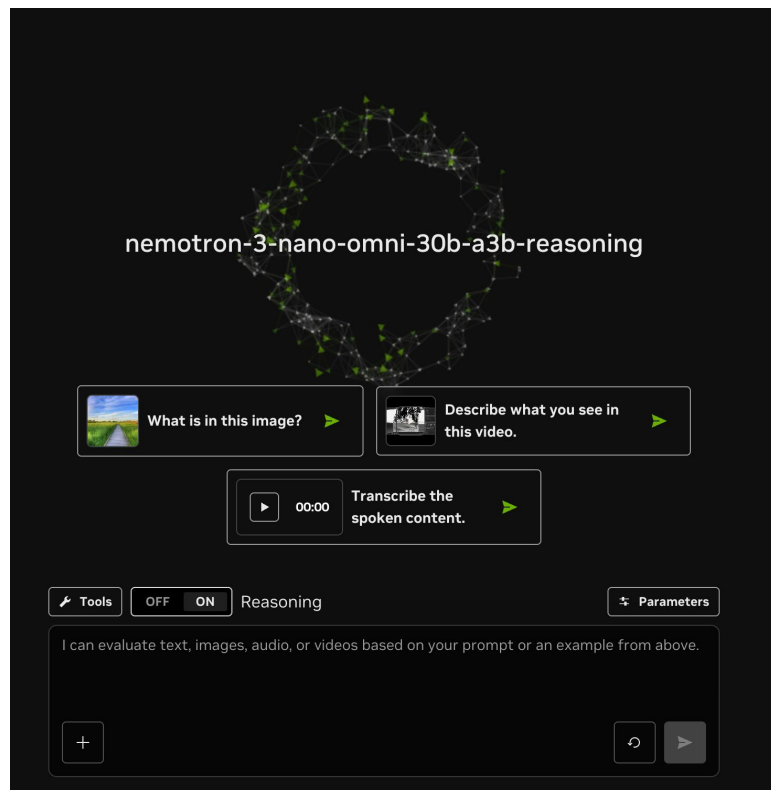
- [Configuring your NVIDIA API KEY to use models from build.nvidia.com](#)

Slides



Nemotron 3 Nano Omni

Reasoning VLM trained on tool calling



[nemotron-3-nano-omni-30b-a3b-reasoning](#) (live demo)

[Nemotron 3 Nano Omni: Efficient and Open Multimodal Intelligence](#) (ArXiv)

Describing Multimodal Data with Nemotron 3 Nano Omni

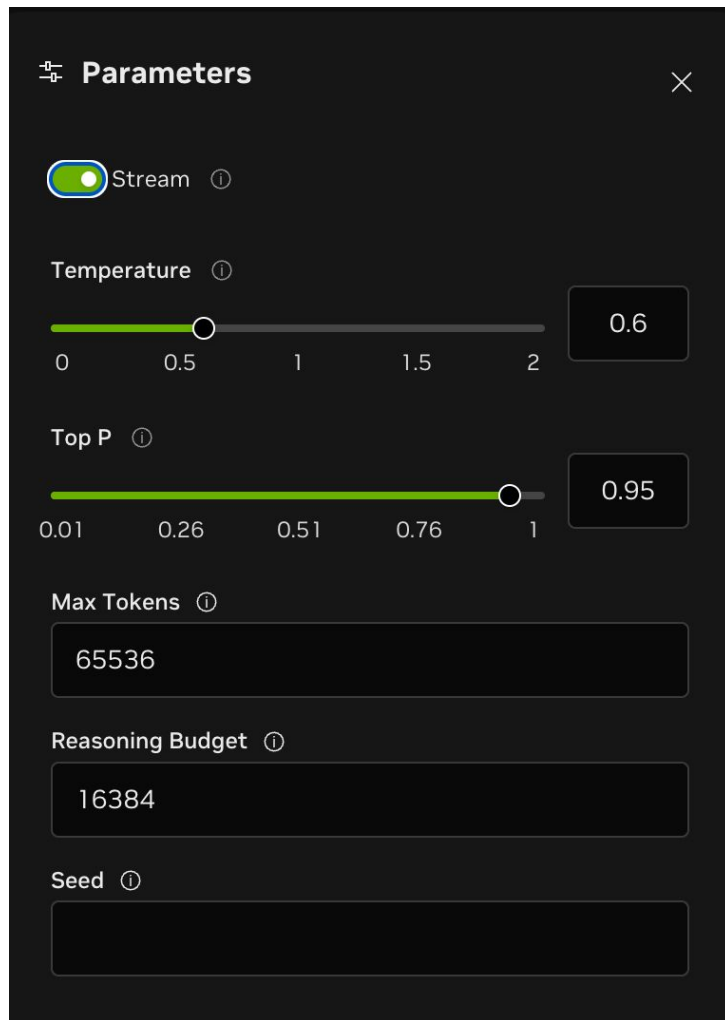


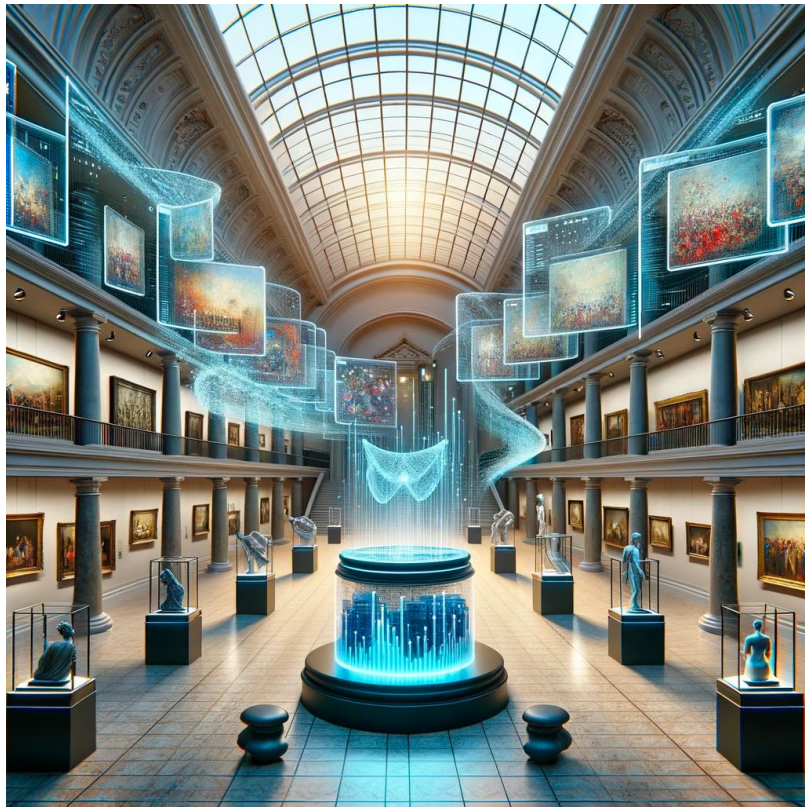
*"A galloping
horse, seen
from the side,
with the jockey
riding it"*

Understand the architecture to understand its parameters



<https://build.nvidia.com/nvidia/nemotron-3-nano-omni-30b-a3b-reasoning>



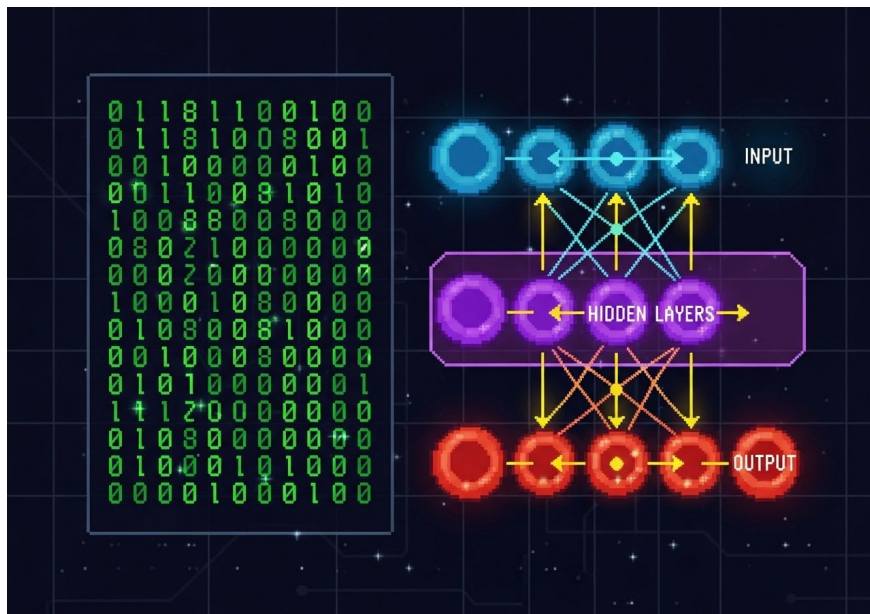


Use cases

- Video search and summarization 🎬
- Multimodal document understanding 📖
- Audio transcription 🗣️
- Dataset curation 🔧

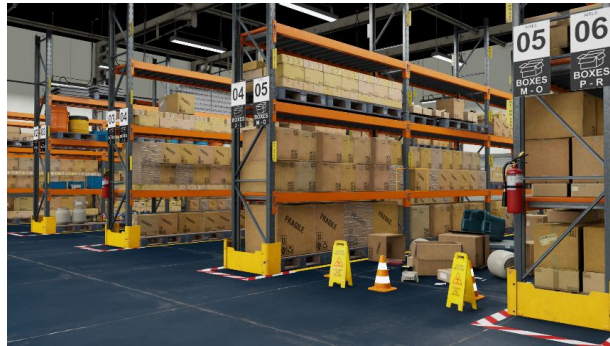
Working Hypothesis

Any data source that can be turned into a matrix can be used to train and run inference on a neural network

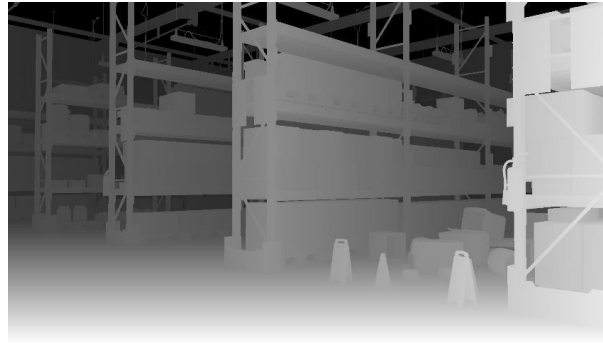


Sight

How A Computer Might See



RGB



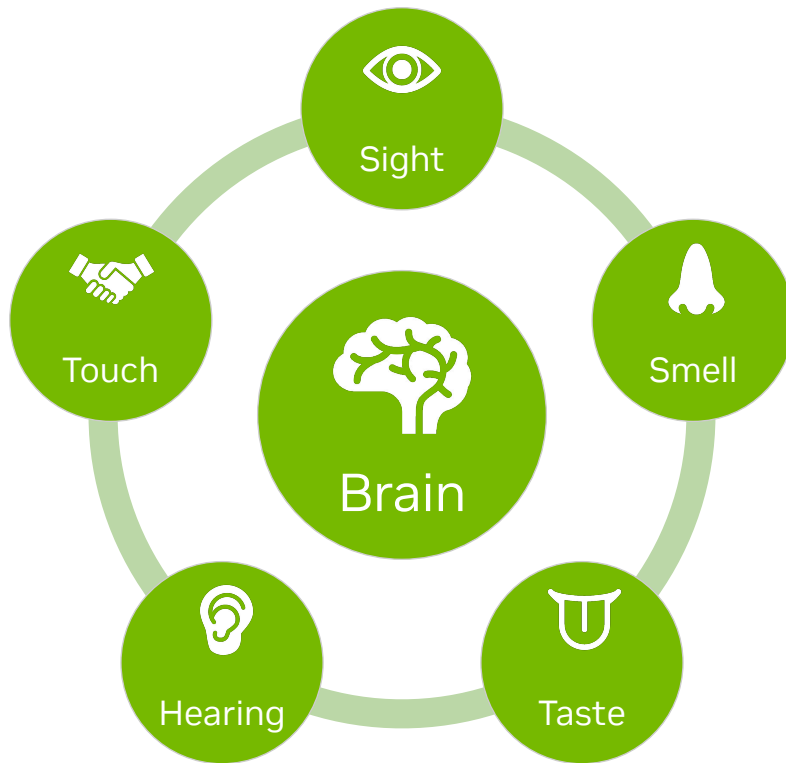
Depth



Fisheye

The Five Senses

How Humans Perceive



Smell and Taste

How A Computer Might Smell and Taste

pH



Nutrition Facts

Nutrition Facts	
1 serving per container	
Serving size	140 g
Amount per serving	
Calories	100
% Daily Value *	
Total Fat 0.5g	1%
Saturated Fat 0g	0%
Trans Fat 0g	
Cholesterol 0mg	0%
Sodium 0mg	0%
Total Carbohydrate 24g	9%
Dietary Fiber 6g	22%
Total Sugars 15g	
Includes 0g Added Sugars	0%
Protein <1g	
Vitamin D 0mcg	0%
Calcium 0mg	0%
Iron 0.3mg	2%
Potassium 120mg	2%
* The % Daily Value (DV) tells you how much a nutrient in a serving of food contributes to a daily diet. 2,000 calories a day is used for general nutrition advice.	

Molecular Weight



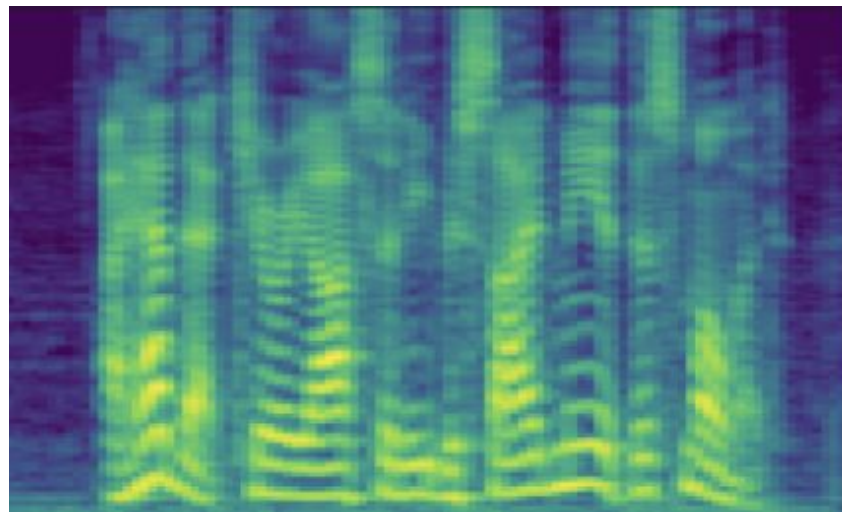
Hearing

How A Computer Might Hear

Sheet Music



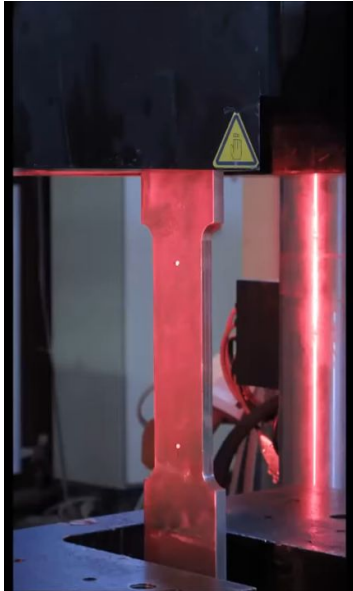
Spectrogram



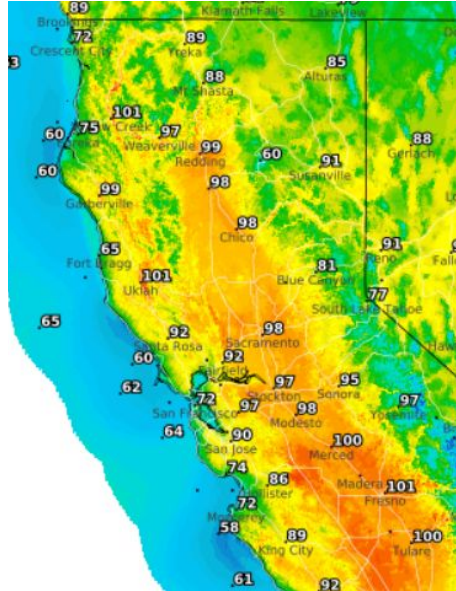
Touch

How A Computer Might Feel

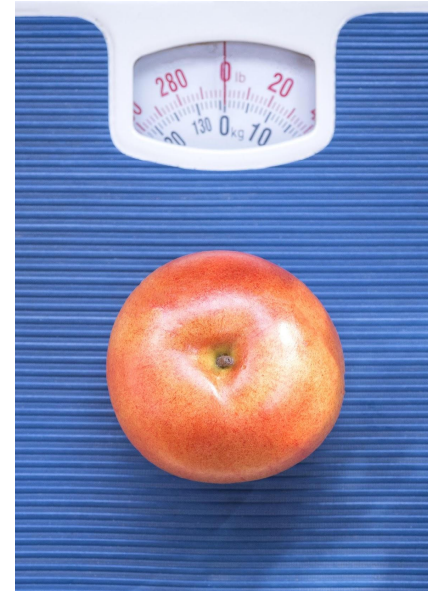
Tensile Strength



Temperature

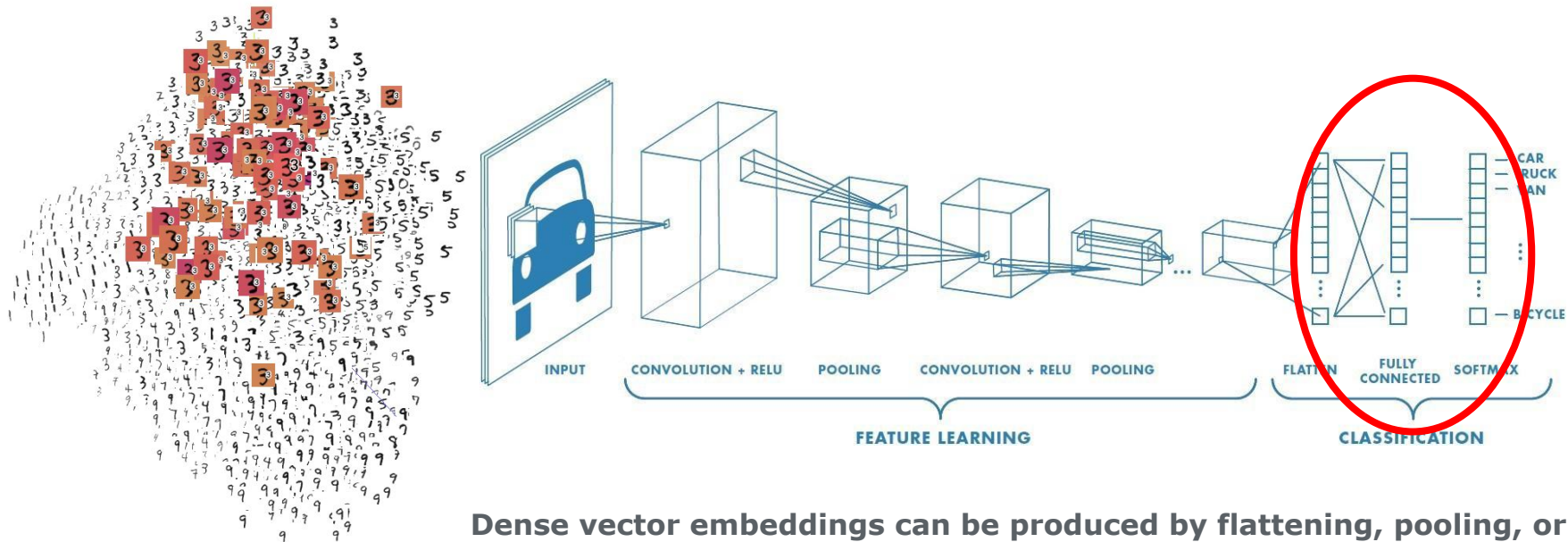


Weight



Embeddings as the enabler of multimodality

When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other



Dense vector embeddings can be produced by flattening, pooling, or feeding (learned) features to linear layers

[Source of image](#) (try exploring interactively)

Sparse vs dense vector representations

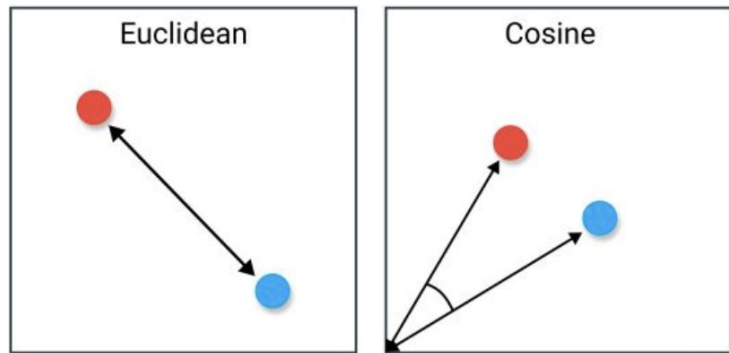
$$\mathbf{x}_1 = \text{[Image of handwritten 3]} \quad \mathbf{x}_2 = \text{[Image of handwritten 3]} \quad 28 \times 28 \text{ pixels} = 784 \text{ values}$$

$$\mathbf{x}_1 \text{ sparse} = \mathbf{x}_2 \text{ sparse} = [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^\top$$

$$\mathbf{x}_2 \text{ dense} = [0.10 \quad -0.75 \quad 0.35 \quad 0.41 \quad -0.20 \quad 0.89 \quad -0.60 \quad 0.03 \quad 0.59 \quad -0.50]^\top$$

$$\mathbf{x}_1 \text{ dense} = [0.12 \quad -0.85 \quad 0.37 \quad 0.44 \quad -0.22 \quad 0.90 \quad -0.63 \quad 0.01 \quad 0.58 \quad -0.49]^\top$$

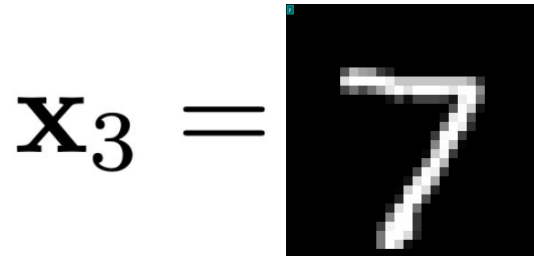
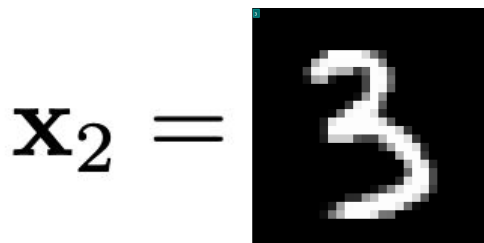
Common metrics to compare embeddings



$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

Similar objects have similar embeddings



Similar digits (X_1, X_2): High cos similarity (≈ 0.98), Low Euclidean distance (≈ 0.3)

Different digits (X_1, X_3): Low cos similarity (≈ 0.4), High Euclidean distance (≈ 2.1)

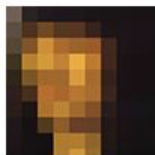
README.md

Art Recommendation system

This is the repository of a portfolio project at [DSR](#). This project aims to identify similar images using pre-trained computer vision networks. For an explanation of the technology see the [technology section](#).

Contributors

- [Catarina Ferreira](#)
- [Gargi Maheshwari](#)

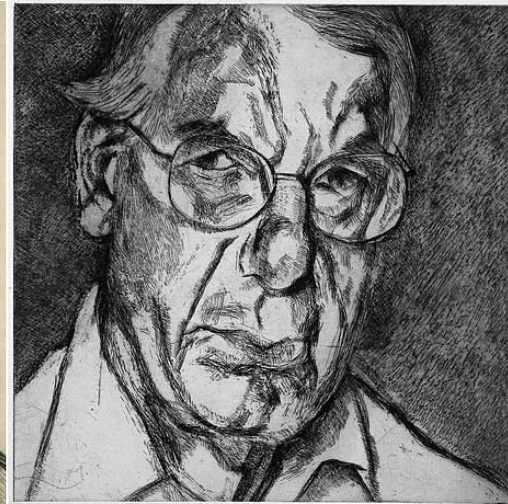


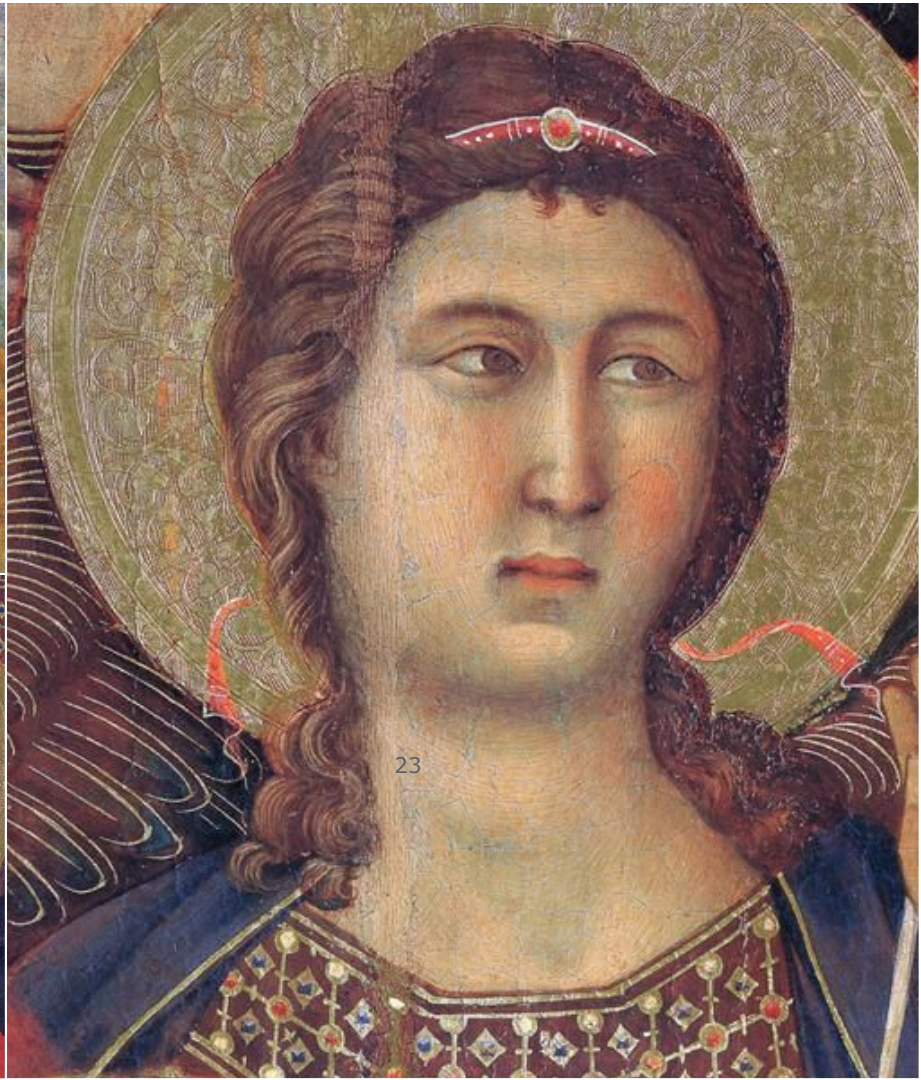
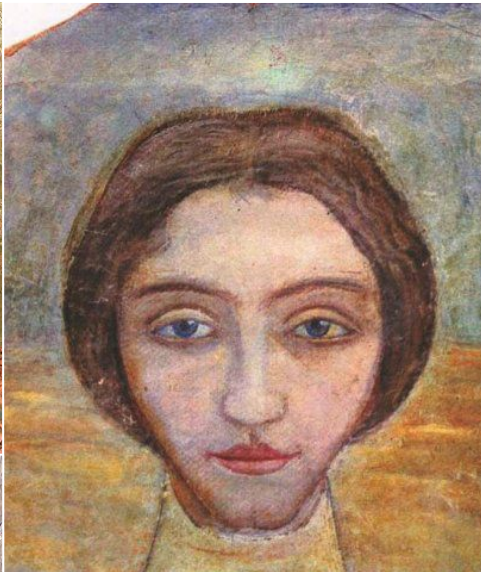
WIKIART
VISUAL ART ENCYCLOPEDIA

<https://github.com/gargimaheshwari/Wikiart-similar-art>

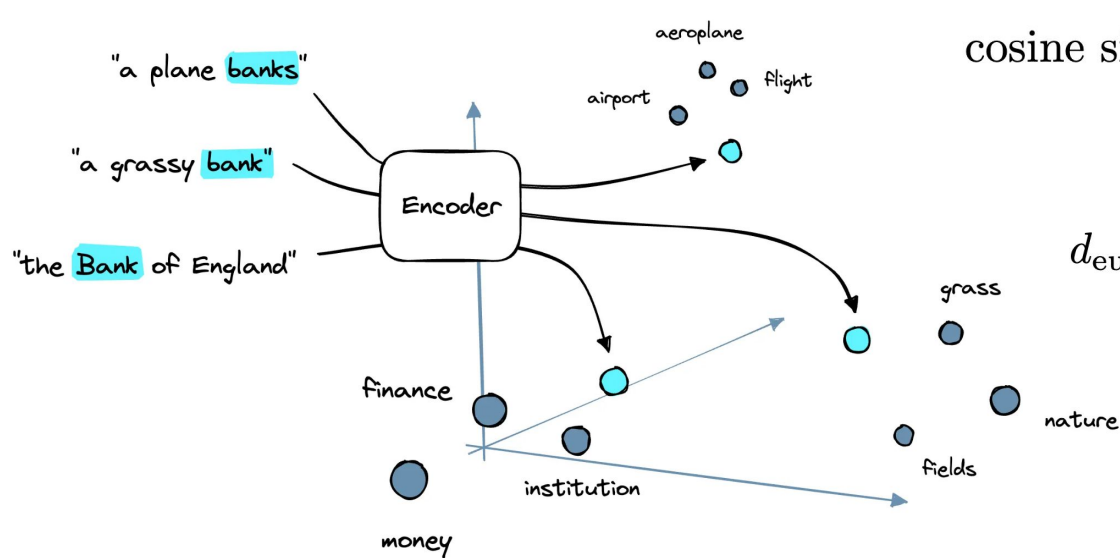








Text encoders

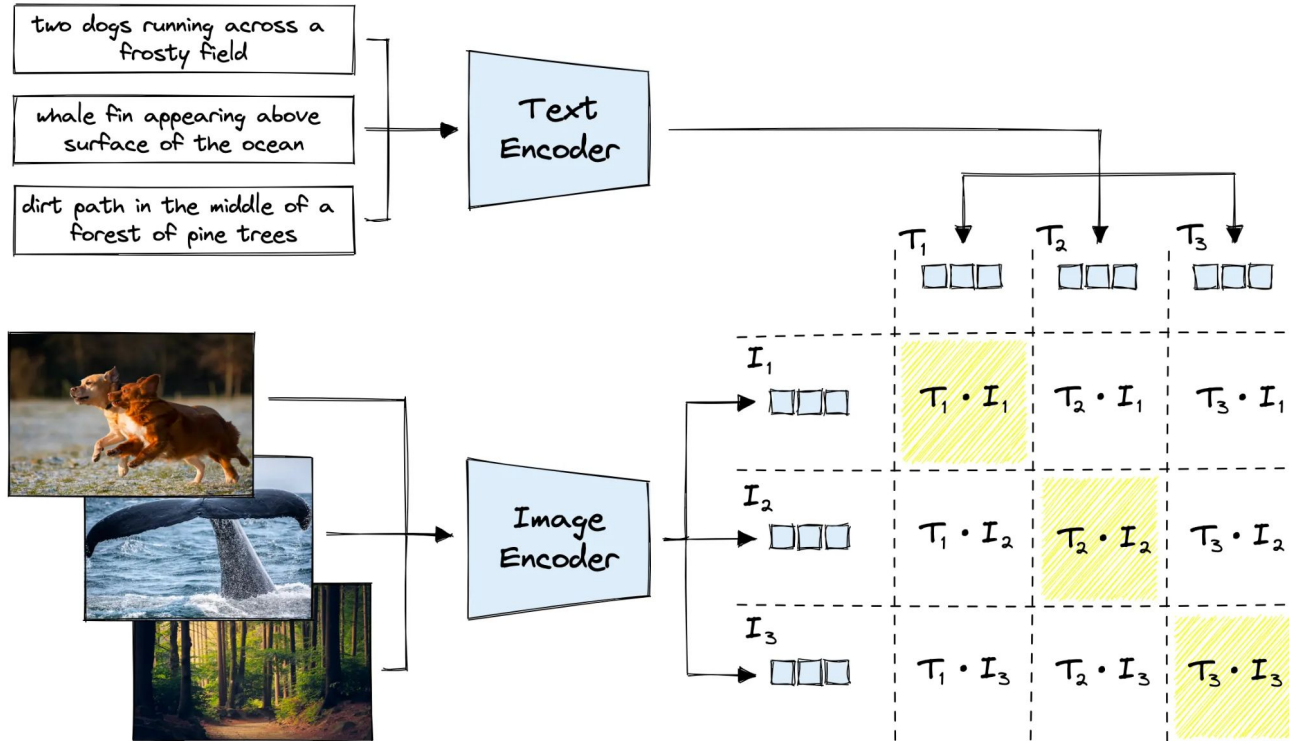


$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

Image from <https://www.pinecone.io/learn/series/image-search/vision-transformers/>

CLIP training



NVIDIA's C-RADIO for Text ↔ Image

Text Prompt: "shoe"



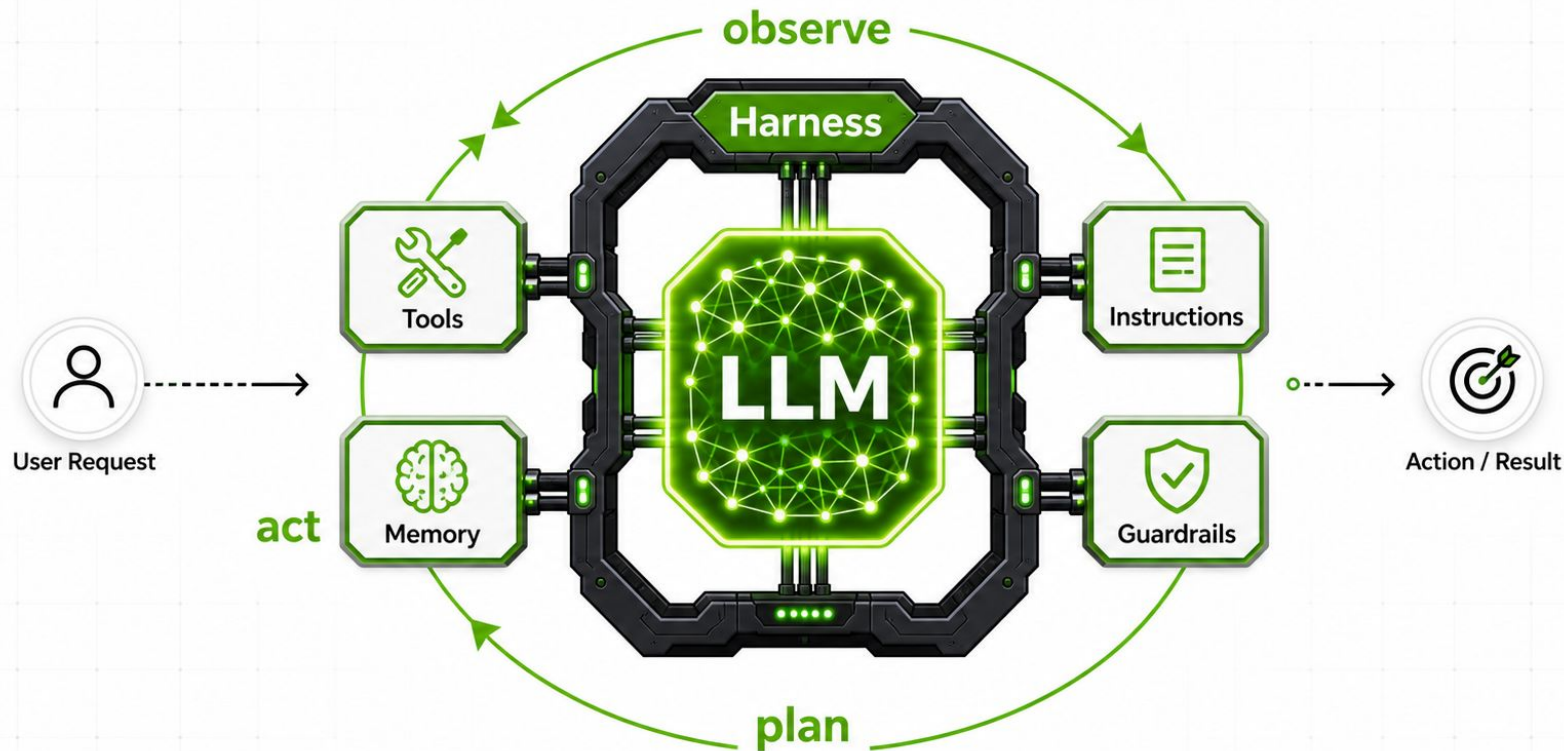
Text Prompt: "helmet"



<https://huggingface.co/nvidia/C-RADIOv4-H>

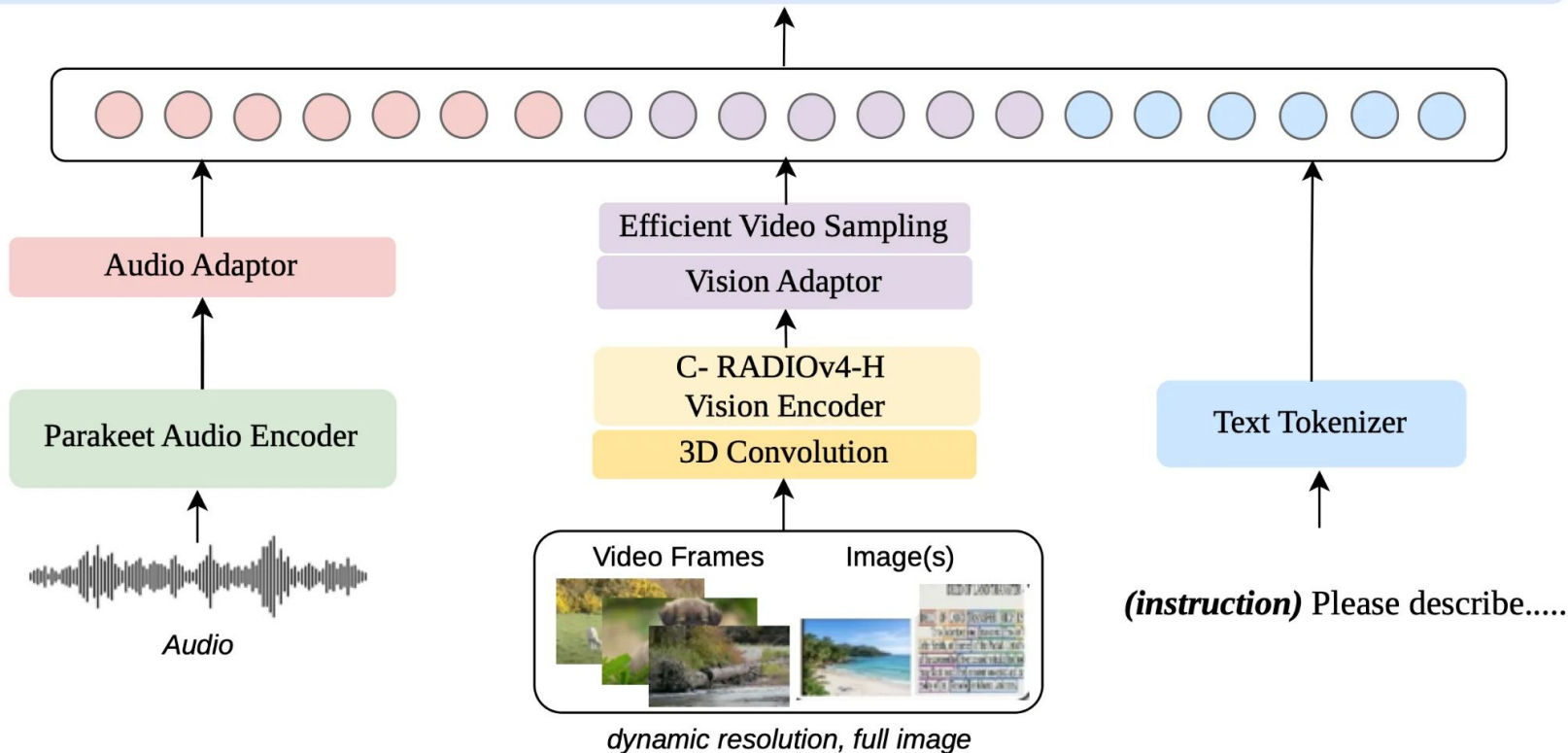
[C-RADIOv4 \(Tech Report\)](#)

AI Agent = LLM + Harness



There is a

Nemotron 3 Nano 30B-A3B LLM



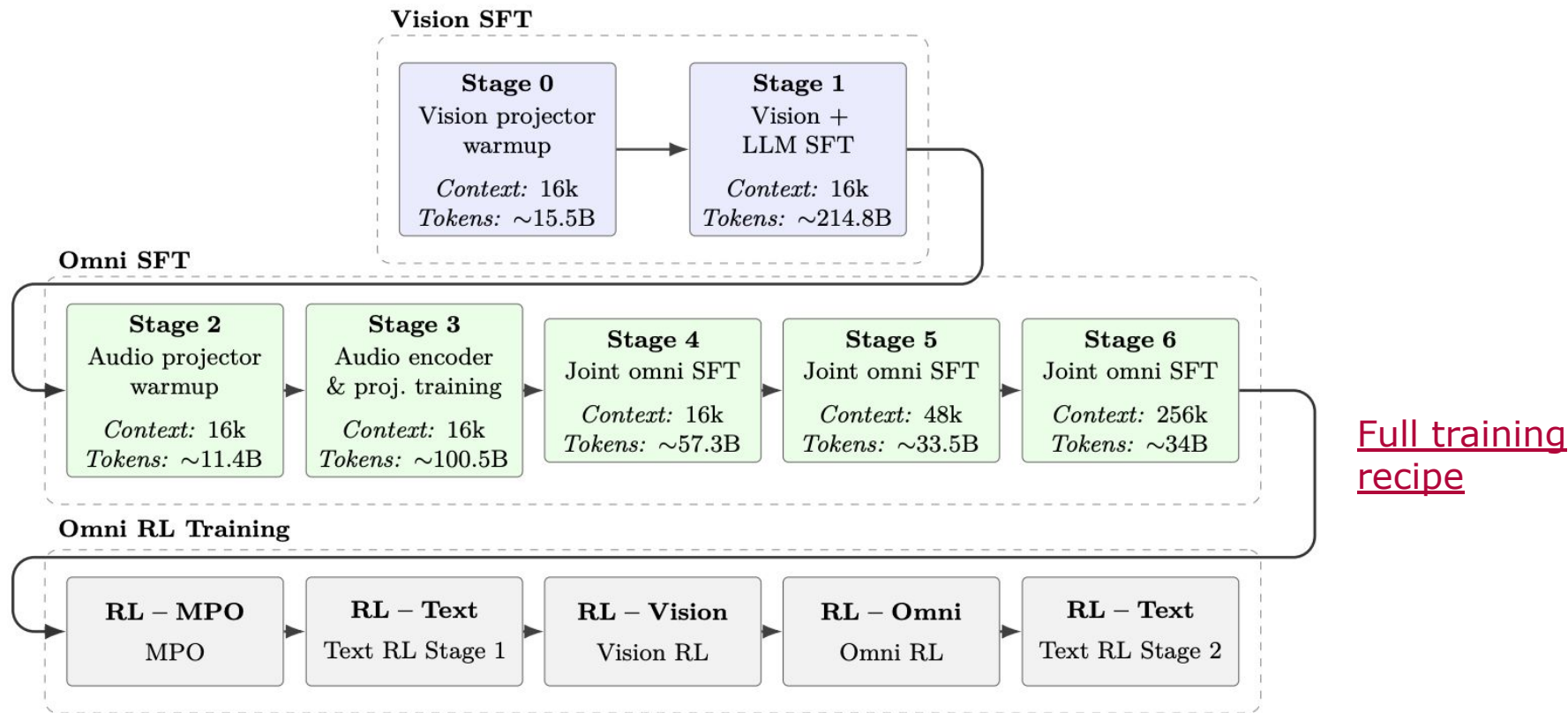


Figure 2 | Staged training recipe for the v3 omni-modal model. The pipeline first performs vision SFT, then joint omni SFT while progressively extending context length, followed by omni-modal RL training.

Text Generation with Large Language Models

Auto-regressive prediction of tokens



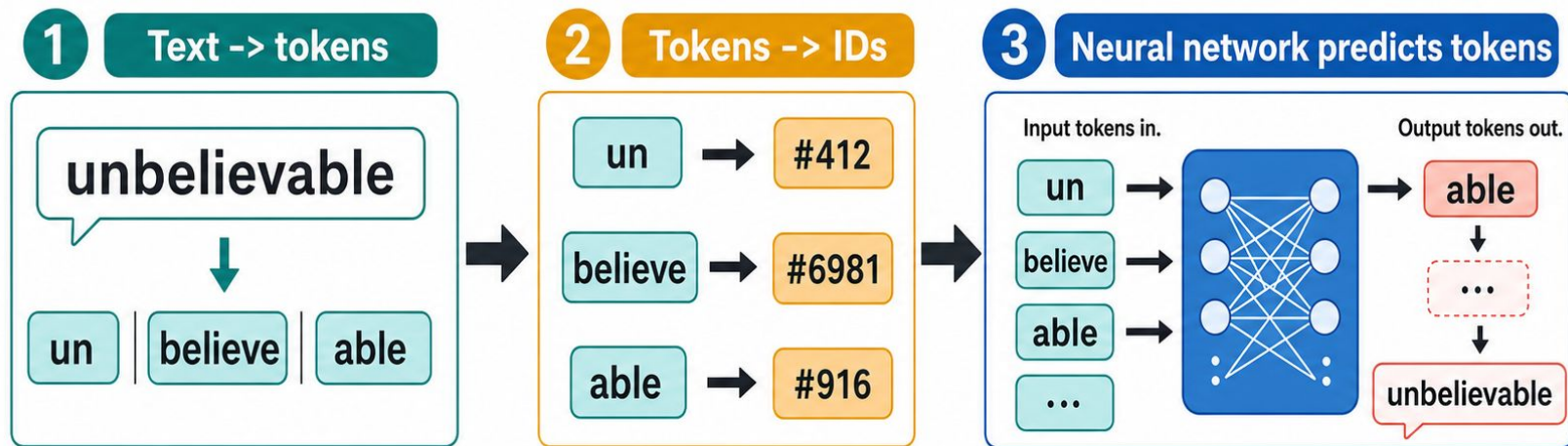
[Deep Dive into LLMs like ChatGPT](#)

What is a token?

A token is the unit of text (group of words, word, subword, or character) that is mapped to an integer ID and used as the input and output of the model.

Tokens are how LLMs read and write

Indexed subwords are both the input and output of the neural network.



Example 1: unbelievable -> un | believe | able

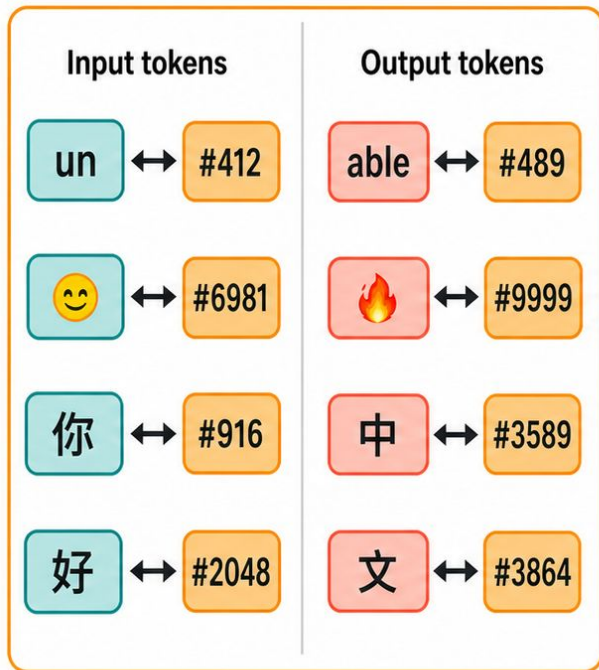
Example 2: un = #412 believe = #6981 able = #916

Caption: Input tokens in. Output tokens out.

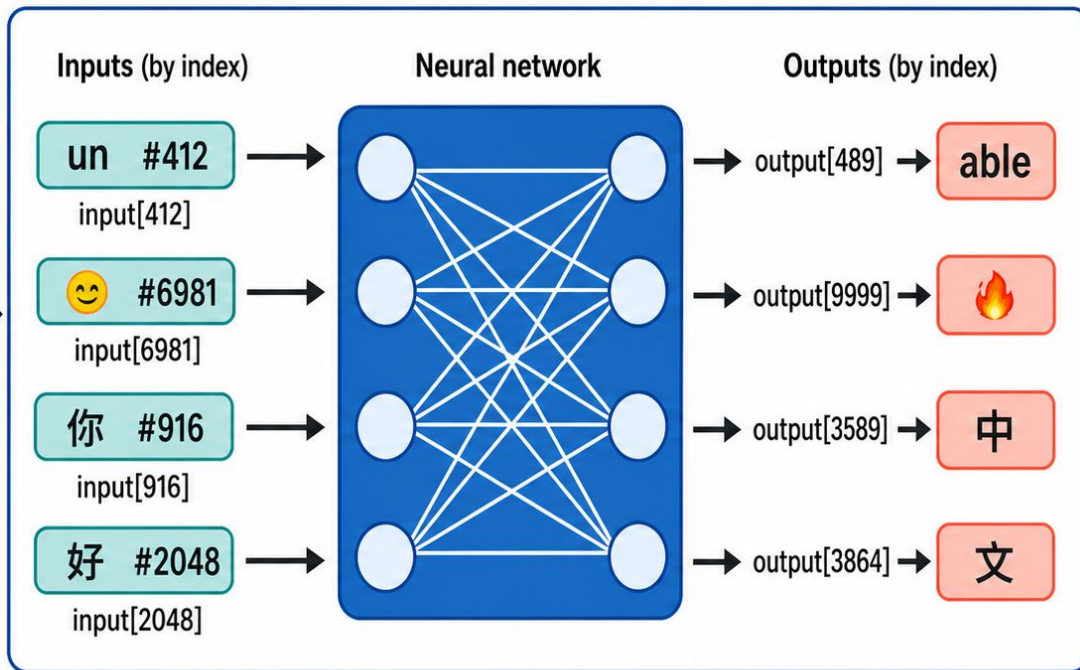
LLMs do not process words directly; they process token indexes.

[Let's reproduce GPT-2 \(124M\)](#)

Tokens ↔ IDs

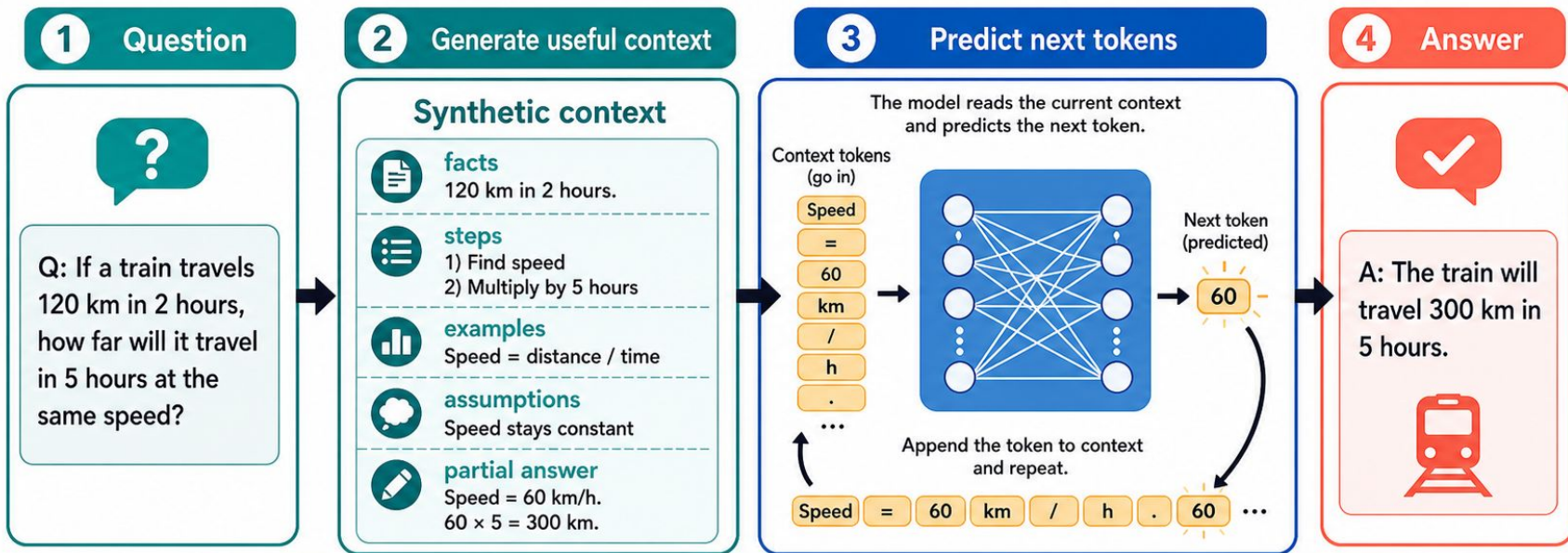


Token IDs map to neural network units by index



How do LLMs "reason"?

They produce synthetic context before answering.



Reasoning is more generated text: intermediate tokens that shape the final output.

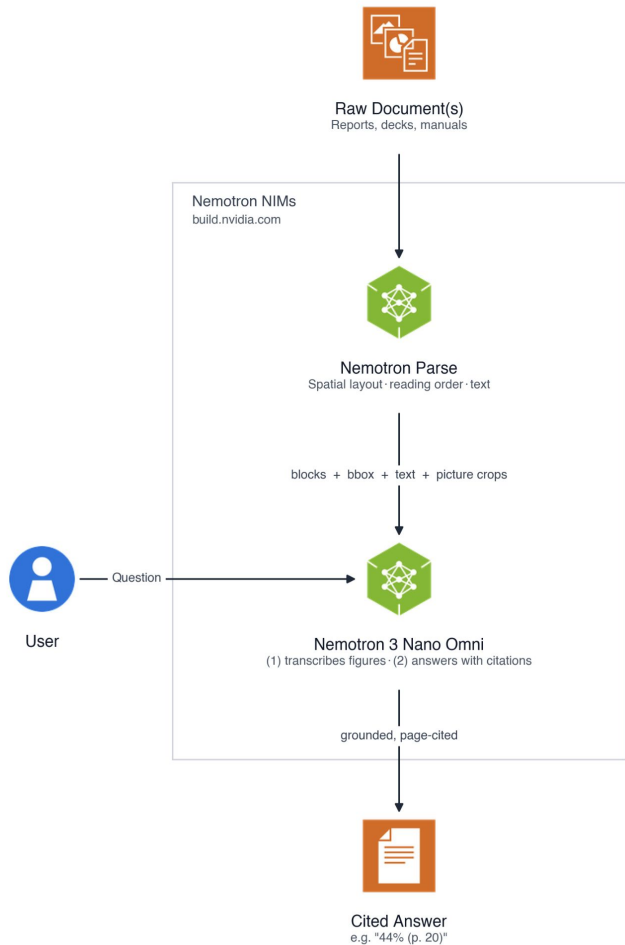


The model is not pausing to think like a person; it is extending context token by token.

[Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)



Would you call this “multi-modal”?



For complex OCR it's best to pair with Nemotron Parse

4.5. Reasoning budget control

We study the effect of inference-time reasoning budgets by evaluating model performance under two settings: (1) a base configuration with a maximum sequence length of 16,384 tokens, and (2) a reasoning-enabled configuration with a 13K reasoning budget, a 1,024-token grace period, and a maximum sequence length of 16,384 tokens (Table 11).

Our results suggest that reasoning budget adjustment yields accuracy gains on select benchmarks under reasoning-on mode, with no degradation observed on the remaining ones. These gains with budget control may arise from the early termination of malformed reasoning traces with repetition loops on out-of-distribution tasks, as well as the truncation of overly verbose reasoning chains for problems requiring minimal or straightforward reasoning.

Benchmark	MathVista-Mini	MMLongBench-Doc	DocVQA (Val)	Charxiv(RQ)	RefCOCO	VideoMME
w.o. reasoning budget	80.3	54.5	95.3	61.8	90.4	67.5
w. reasoning budget	82.8	56.8	95.2	64	90.6	70.3

Table 11 | Effect of reasoning budget across several key benchmarks.

Nemotron Parse

Creates additional context for Nemotron 3 Omni

nvidia

nemotron-parse [Downloadable](#)

Cutting-edge vision-language model excelling in retrieving text and metadata from images.

[document parsing](#) [supported language - english](#) [text and table extraction](#)

[Get API Key](#)

[Experience](#) [Model Card](#) [Deploy](#)

[API Reference](#) [Accelerated by DGX Cloud](#)

AI models generate responses and outputs based on complex algorithms and machine learning techniques, and those responses or outputs may be inaccurate, harmful,...

Input [Try](#) [Python](#) [Shell](#)

Upload Image [View Examples](#)

CUDA C++ Programming Guide, Release 12.8

Thread Scope	Description
cuda::thread_scope::thread_scope_thread	Only the CUDA thread which initiated asynchronous operations synchronizes.
cuda::thread_scope::thread_scope_block	All or any CUDA threads in the same GPU block as the initiating thread synchronizes.
cuda::thread_scope::thread_scope_device	All or any CUDA threads in the same GPU device as the initiating thread synchronizes.
cuda::thread_scope::thread_scope_system	All or any CUDA or CPU threads in the same system as the initiating thread synchronizes.

These thread scopes are implemented as extensions to standard C++ in the [CUDA Standard C++ library](#).

5.6. Compute Capability

The compute capability of a device is represented by a version number (also sometimes called the "SM version"). The version number identifies the features supported by the GPU hardware and is used for determining if certain features are supported by a given hardware device. Compute capabilities are available on the [compute capability](#) page.

The compute capability comprises a major version number *X* and a minor version number *Y* and is denoted as *CC_X.Y*.

Devices with the same major version number are of the same architecture. The major version number is the device based on the CUDA major release. The device based on the hardware architecture is the device based on the hardware architecture. The device based on the hardware architecture is the device based on the hardware architecture.

GOVERNING TERMS: Access to this model is governed by the [NVIDIA API Trial Terms of Service](#); use of the model is governed by the [NVIDIA Community Model License](#). Use of the tokenizer included in this model is governed by the [CC-BY-4.0 license](#).

[Reset](#) [Run](#)

Output [Preview](#) [JSON](#)

Detected Elements

Page-header

CUDA C++ Programming Guide, Release 12.8

Table

Thread Scope	Description
cuda::thread_scope::thread_scope_thread	Only the CUDA thread which initiated asynchronous operations synchronizes.
cuda::thread_scope::thread_scope_block	All or any CUDA threads within the same thread block as the initiating thread synchronizes.
cuda::thread_scope::thread_scope_device	All or any CUDA threads in the same GPU device as the initiating thread synchronizes.
cuda::thread_scope::thread_scope_system	All or any CUDA or CPU threads in the same system as the initiating thread synchronizes.

Text

These thread scopes are implemented as extensions to standard C++ in the [CUDA Standard C++ library](#).

Section-header

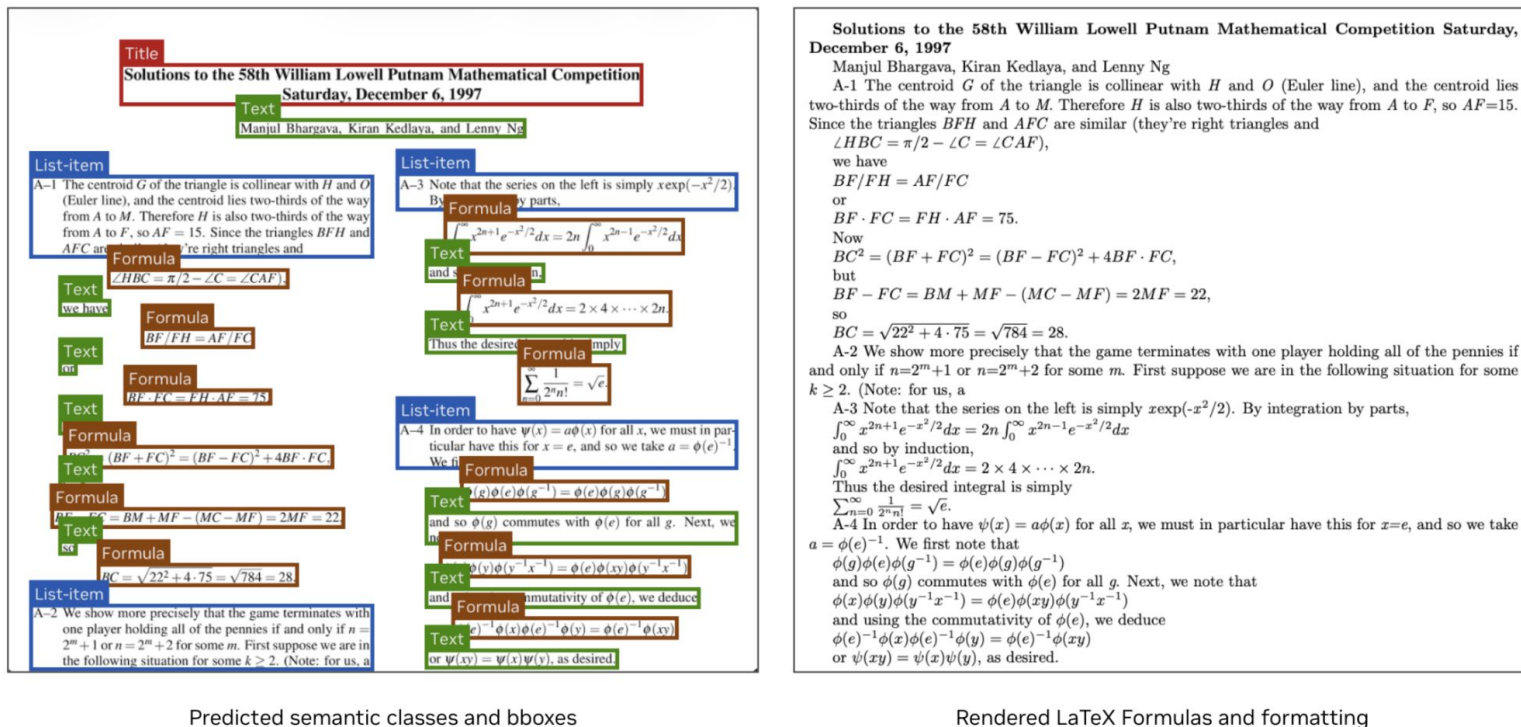
5.6. Compute Capability

Text

The compute capability of a device is represented by a version number (also sometimes called the "SM version").

<https://build.nvidia.com/nvidia/nemotron-parse>

 NVIDIA



Predicted semantic classes and bboxes

Rendered LaTeX Formulas and formatting

Figure 2 | OCR, extraction of text formatting and mathematical equations in LaTeX and markdown.

2.2.2. Output format

Nemotron-Parse predicts bounding boxes of the semantic blocks in the form of relative coordinates, in a scale of 1024×1280 . These bounding boxes are predicted in a canonical reading order, that includes Page-Header elements at the start of the page, followed by Text, Section-Header, List-Item, Title and Formula elements in the order as they would be read by a person looking at the given page, followed by Footnotes, Page-Footers, Tables, Pictures, and Captions in the end. Nemotron-Parse-TC improves upon this canonical reading order, and also includes non-reading-order (floating) elements, i.e., Footnotes, Page-Footers, Tables, Pictures, and Captions within the natural ordering of the page.

In the maximal information setting the output is in the following format:

```
<x_(\d+)><y_(\d+)>(.*?)<x_(\d+)><y_(\d+)>  
<class_(\w+)>
```

where the **first group** denotes the coordinates of the top-left corner, the **second group** denotes the text contained within the bounding box, the **third group** denotes the coordinates of the bottom-right corner, and the **final group** represents the semantic class. For example, `<x_0.1152><y_0.2586># NVIDIA Nemotron-Parse 1.1<x_0.8799><y_0.2797> <class_Title>`.

Summary

Nemotron 3 Omni unifies text, vision, and audio in a single model

- Embeddings enable flexible multimodal workflows
- Two modes: descriptive and reasoning

Powers end-to-end pipelines for document intelligence, multimodal search, and assistants

- Multimodal prompts let the model attend to specific regions, layouts, or visual details
- Integrates with tools like Nemotron Parse and RAG to build production-ready multimodal apps

Resources

- [Nemotron 3 Nano Omni - Technical Report](#)
- [Nemotron Developer repository at GitHub](#)
- [Nemotron 3 Nano Omni 30B @ HuggingFace](#)
- [Nemotron Parse 1.2 @ HuggingFace](#)
- [Practical Computer Vision Bootcamp - GitHub Repository](#)

Grow Your Skills with NVIDIA Learning Paths

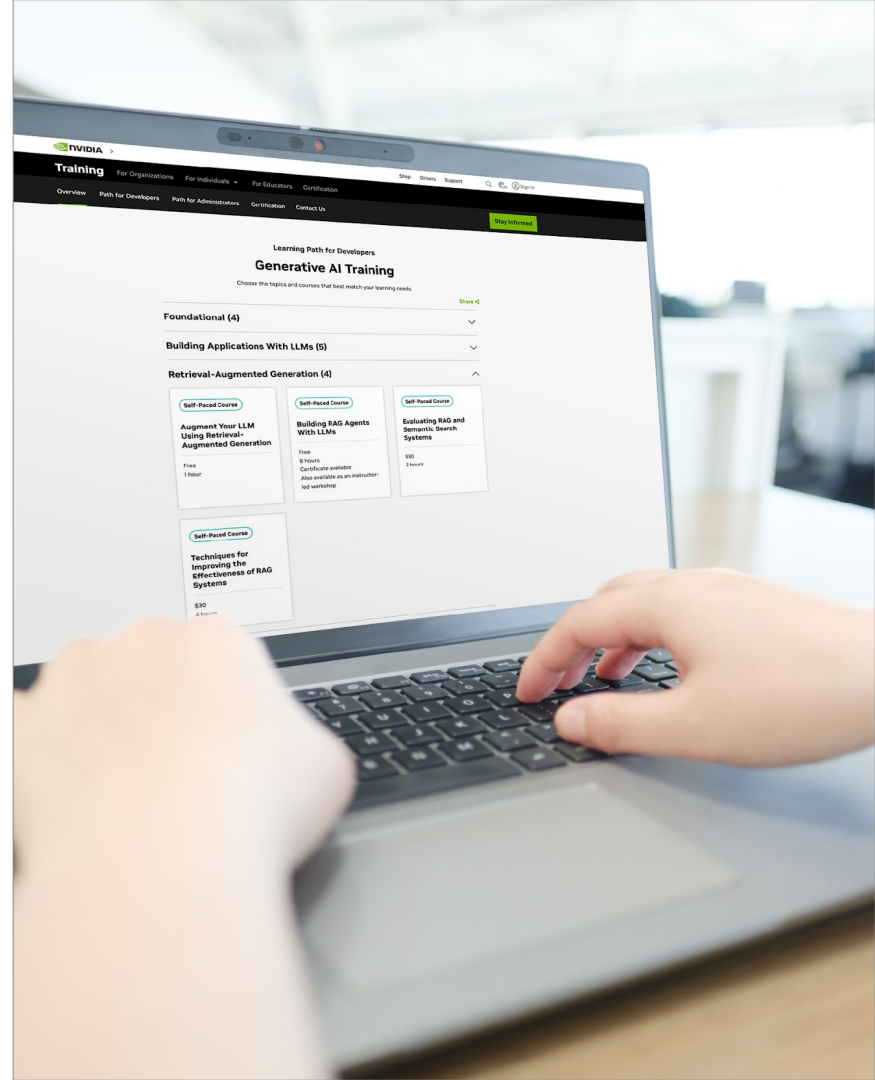
Build expertise in AI, accelerated computing, and graphics through guided NVIDIA learning paths

High-Level Benefits

- Follow structured, role-based skill development
- Learn through hands-on labs and real-world projects
- Earn industry-recognized certifications



Learn More and Apply
<https://nvda.ws/3ZXQYb7>



GTC Is Coming to Berlin



Join NVIDIA GTC is Berlin - where developers, researchers, and business leaders will explore technologies transforming every industry.

It all starts with CEO Jensen Huang's keynote at the iconic Tempodrom, followed by sessions, hands-on labs, and networking.

Registration is now open.

Workshops October 20, 2026

Conference and Expo October 21-22



nvidia.com/en-eu/gtc

